# WEEK-7
# Programs on 1D arrays, identifying device properties in CUDA

Niharika Gawade
240905284
Roll no:30
CSE-D

**1. Implement CUDA program to determine the number of CUDA capable devices in the system. For each device, examine the following device's properties:**

- **a. Device name**
- **b. Number of Streaming Multiprocessors**
- **c. Maximum number of threads per block**
- **d. Maximum thread dimensions (x,y,z)**
- **e. Maximum grid dimensions (x,y,z)**
- **f. The device clock frequency.**

```
#include <stdio.h>
#include <cuda_runtime.h>

int main() {
   int n;
   cudaGetDeviceCount(&n);
   printf("CUDA Devices: %d\n", n);

   for (int i = 0; i < n; i++) {
      cudaDeviceProp p;
      cudaGetDeviceProperties(&p, i);

      printf("\nDevice %d: %s\n", i, p.name);
      printf("SMs: %d\n", p.multiProcessorCount);
      printf("Max Threads/Block: %d\n", p.maxThreadsPerBlock);
      printf("Max Threads Dim: %d %d %d\n",
           p.maxThreadsDim[0], p.maxThreadsDim[1], p.maxThreadsDim[2]);
      printf("Max Grid Dim: %d %d %d\n",
           p.maxGridSize[0], p.maxGridSize[1], p.maxGridSize[2]);
      printf("Clock Frequency: %d kHz\n", p.clockRate);
   }
   return 0;
}
```

**OUTPUT**

```
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_7$ nvcc p1.cu -o p1
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_7$ ./p1
CUDA Devices: 1

Device 0: NVIDIA GeForce GT 1030
SMs: 3
Max Threads/Block: 1024
Max Threads Dim: 1024 1024 64
Max Grid Dim: 2147483647 65535 65535
Clock Frequency: 1468000 kHz
```

**2. Given two vectors of length N, implement a program in CUDA that launches two kernel to do the following tasks**
**a) Add two vectors (kernel execution configuration: Block size as N)**
**b) Dot product of two vectors (kernel execution configuration: N threads)**

```
#include <stdio.h>
#include <cuda_runtime.h>

__global__ void add(float *a, float *b, float *c) {
    int i = threadIdx.x;
    c[i] = a[i] + b[i];
}

__global__ void dot(float *a, float *b, float *r) {
    int i = threadIdx.x;
    r[i] = a[i] * b[i];
}

int main() {
    int N = 10;
    float a[10], b[10], c[10], r[10];
    float *da, *db, *dc, *dr;

    for (int i = 0; i < N; i++) {
        a[i] = i;
        b[i] = i * 2;
    }

    cudaMalloc(&da, N*sizeof(float));
    cudaMalloc(&db, N*sizeof(float));
    cudaMalloc(&dc, N*sizeof(float));
    cudaMalloc(&dr, N*sizeof(float));

    cudaMemcpy(da,a,N*sizeof(float),cudaMemcpyHostToDevice);
    cudaMemcpy(db,b,N*sizeof(float),cudaMemcpyHostToDevice);
```

```
    add<<<1,N>>>(da,db,dc);
    dot<<<1,N>>>(da,db,dr);

    cudaMemcpy(c,dc,N*sizeof(float),cudaMemcpyDeviceToHost);
    cudaMemcpy(r,dr,N*sizeof(float),cudaMemcpyDeviceToHost);

    printf("a   b    Add      Dot    \n");
    for(int i=0;i<N;i++){
          a[i] = i;
       b[i] = i * 2;
       printf("%.2f  %.2f  %.2f  %.2f \n",a[i],b[i],c[i],r[i]);
     }

    cudaFree(da); cudaFree(db); cudaFree(dc); cudaFree(dr);
    return 0;
}
```

**OUTPUT**

```
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_7$ nvcc p2.cu -o p2
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_7$ ./p2
a    b     Add        Dot
0.00  0.00  0.00  0.00
1.00  2.00  3.00  2.00
2.00  4.00  6.00  8.00
3.00  6.00  9.00  18.00
4.00  8.00  12.00  32.00
5.00  10.00  15.00  50.00
6.00  12.00  18.00  72.00
7.00  14.00  21.00  98.00
8.00  16.00  24.00  128.00
9.00  18.00  27.00  162.00
```

**3. Implement a CUDA program to compute the Euclidean distance between two randomly initialized vectors A and B, each of length N. Keep the number of threads per block fixed at 256 and dynamically determine the number of blocks required to process all N elements.**

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <cuda_runtime.h>

__global__ void distance(float *a, float *b, float *c, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if(i < n)
        c[i] = (a[i]-b[i])*(a[i]-b[i]);
}

int main() {
    int N = 1000;
```

```
    float *a,*b,*c,*d;
    float sum=0;

    a=malloc(N*sizeof(float));
    b=malloc(N*sizeof(float));
    c=malloc(N*sizeof(float));

    for(int i=0;i<N;i++) {
        a[i]=(float)rand()/RAND_MAX;
        b[i]=(float)rand()/RAND_MAX;
    }

    cudaMalloc(&d,N*sizeof(float));
    float *da,*db;
    cudaMalloc(&da,N*sizeof(float));
    cudaMalloc(&db,N*sizeof(float));

    cudaMemcpy(da,a,N*sizeof(float),cudaMemcpyHostToDevice);
    cudaMemcpy(db,b,N*sizeof(float),cudaMemcpyHostToDevice);

    distance<<<(N+255)/256,256>>>(da,db,d);
    cudaMemcpy(c,d,N*sizeof(float),cudaMemcpyDeviceToHost);

    for(int i=0;i<N;i++) sum+=c[i];

    printf("Euclidean Distance = %f\n",sqrt(sum));

    cudaFree(da); cudaFree(db); cudaFree(d);
    free(a); free(b); free(c);
    return 0;
}
```

**OUTPUT**

```
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_7$ nvcc p3.cu -o p3
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_7$ ./p3
Euclidean Distance = 12.630070
```

**4. Implement a program in CUDA to process a 1D array containing angles in radians to generate sine of the angles in the output array. Use appropriate function.**

```
#include <stdio.h>
#include <cuda_runtime.h>
#include <math.h>

__global__ void sine(float *a, float *b, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if(i < n)
        b[i] = sinf(a[i]);
}

int main() {
```

```
    int N = 10;
    float a[10], b[10];
    float *da,*db;

    for(int i=0;i<N;i++)
        a[i] = i * 0.5f;

    cudaMalloc(&da,N*sizeof(float));
    cudaMalloc(&db,N*sizeof(float));

    cudaMemcpy(da,a,N*sizeof(float),cudaMemcpyHostToDevice);

    sine<<<(N+255)/256,256>>>(da,db,N);

    cudaMemcpy(b,db,N*sizeof(float),cudaMemcpyDeviceToHost);

    for(int i=0;i<N;i++)
        printf("%f -> %f\n",a[i],b[i]);

    cudaFree(da);
    cudaFree(db);
    return 0;
}
```

**OUTPUT**

```
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_7$ nvcc p4.cu -o p4
5CSED1@dsl-11:~/Desktop/Niharika_240905284/WEEK_7$ ./p4
0.000000 -> 0.000000
0.500000 -> 0.479426
1.000000 -> 0.841471
1.500000 -> 0.997495
2.000000 -> 0.909297
2.500000 -> 0.598472
3.000000 -> 0.141120
3.500000 -> -0.350783
4.000000 -> -0.756802
4.500000 -> -0.977530
```